



Phase 2 Complete - Booking Service

Phase 2 has been successfully implemented! Your Booking Service application now includes the complete booking workflow, client portal, email reminders, and calendar view.



What's New in Phase 2

1. Public Booking Flow (`/book/[businessSlug]`)

Route: `/book/luxe-beauty` (or any business slug)

A complete 5-step booking wizard that allows clients to:

- Browse available services with prices and durations
- Select a staff member (or “any available”)
- Pick a date from an interactive calendar
- Choose from real-time available time slots
- Enter contact information and complete booking

Key Features:

- Server-side availability calculation considering:
- Staff weekly schedules (`availability_rules`)
- Date-specific exceptions (holidays, blocked dates)
- Existing bookings
- Staff buffer times
- Slot validation to prevent double-booking
- Mobile-responsive design
- Smooth step-by-step flow with progress indicator

2. Client Portal

Routes:

- `/client/bookings` - View and manage bookings
- `/client/profile` - Edit profile information

Features:

- View upcoming appointments (with confirm/cancel actions)
- View past appointment history
- Confirm pending bookings
- Cancel bookings (with cancellation policy enforcement)
- Update profile (name, phone)
- Dedicated client layout with navigation

3. Email Reminder System

Implementation:

- Resend integration for transactional emails
- React Email templates with professional design
- Secure token-based confirm/cancel links (7-day expiry)

- Email tracking via `reminder_events` table
- Per-business sender email configuration

How to Send Reminders:

```
import { sendBookingReminder } from '@lib/email/send-reminder';

// Send reminder for a booking
await sendBookingReminder(bookingId);
```

Email Features:

- Beautiful HTML email template
- Appointment details (service, staff, date, time, location)
- One-click “Confirm” and “Cancel” buttons
- Secure token verification
- Confirmation/cancellation pages

4. Calendar View (`/app/calendar`)

Features:

- Week view showing 8 AM - 8 PM (13 hours)
- Color-coded bookings by service
- Staff filtering (view all or individual staff)
- Navigation: previous/next week, today button
- Hover for booking details
- Responsive grid layout
- Real-time data loading

Navigation: Added to dashboard sidebar



Setup & Configuration

1. Install New Dependencies

Already done! But for reference:

```
npm install resend @react-email/components
```

2. Environment Variables

Add to your `.env.local` :

```
# Email (Resend)
RESEND_API_KEY=your_resend_api_key_here
DEFAULT_FROM_EMAIL=Nelsobral@gmail.com

# Security (for email token signing)
BOOKING_TOKEN_SECRET=your-random-secret-string
```

Get Resend API Key:

1. Sign up at resend.com (<https://resend.com>)

2. Go to API Keys
3. Create a new key
4. Paste it into `.env.local`

3. Database - Already Set Up!

The Phase 1 migration already created all necessary tables:

- `bookings` - appointment records
- `clients` - client information
- `availability_rules` - weekly schedules
- `availability_exceptions` - blocked dates
- `reminder_events` - email tracking
- `notification_preferences` - per-business settings

No additional migration needed!



How to Use

Test the Public Booking Flow

1. **Visit the public booking page:**

`http://localhost:3000/book/luxe-beauty`

2. **Complete the booking wizard:**

- Select a service (e.g., "Haircut & Style")
- Choose a staff member (Sophie Laurent or Marcus Chen)
- Pick a date
- Select an available time slot
- Enter your information (name, email, phone)
- Confirm booking

3. **Check the database:**

- A new `booking` record is created
- A new `client` record (if first booking)
- Booking status: `pending`

Test the Client Portal

1. **Log in as a client** (if you have an account linked to a booking)

2. **Visit:**

`http://localhost:3000/client/bookings`

3. **Try actions:**

- Confirm a pending booking
- Cancel a booking (respects cancellation policy)
- Update your profile

Test Email Reminders

Manual Testing (for now):

```
// In a server action or API route
import { sendBookingReminder } from '@lib/email/send-reminder';

export async function testReminder() {
  'use server';
  const bookingId = '66666666-6666-6666-6666-666666666661'; // Use a real booking ID
  const result = await sendBookingReminder(bookingId);
  console.log(result);
}
```

Email Preview:

- Beautiful responsive HTML template
- Appointment details card
- Green “Confirm Appointment” button
- Gray “Cancel Appointment” button
- Professional footer

Test the Links:

1. Click “Confirm Appointment” in the email
2. Should redirect to `/booking-confirmed` page
3. Booking status updated to `confirmed`

Test the Calendar View

1. Go to dashboard:

`http://localhost:3000/app/calendar`

2. Features to try:

- Navigate between weeks
- Filter by staff member
- Click bookings to see details
- View color-coded services



Architecture & Technical Details

Availability Engine

File: `lib/actions/availability.ts`

Algorithm:

1. Get service duration
2. Get staff weekly schedule for selected day
3. Apply date-specific exceptions
4. Get existing bookings for that date
5. Generate 30-minute slots from staff hours
6. Filter out slots that conflict with bookings (including buffer time)
7. Return available slots

Prevents:

- Double bookings
- Booking during blocked dates

- Overlapping appointments
- Buffer time violations

Security

Row Level Security (RLS):

- Clients can only view their own bookings
- Business members can view all business bookings
- Enforced at database level (not just frontend)

Email Token Security:

- HMAC-signed tokens with 7-day expiry
- Includes booking ID + timestamp + signature
- Prevents tampering
- Token verification on every use

Booking Creation:

- Server-side validation
- Race condition protection (double-check availability before insert)
- Concurrency-safe with database constraints

Performance

Server Components:

- Initial data fetched on server
- No client-side loading spinners for first render
- SEO-friendly public booking pages

Client Components:

- Interactive booking wizard
- Real-time slot loading
- Optimistic updates for better UX



Database Schema (Reminder)

Key tables for Phase 2:

```
-- bookings: appointment records
CREATE TABLE bookings (
  id UUID PRIMARY KEY,
  business_id UUID REFERENCES businesses(id),
  client_id UUID REFERENCES clients(id),
  staff_member_id UUID REFERENCES staff_members(id),
  service_id UUID REFERENCES services(id),
  start_time TIMESTAMPTZ NOT NULL,
  end_time TIMESTAMPTZ NOT NULL,
  status booking_status, -- pending, confirmed, cancelled, completed, no_show
  notes TEXT,
  confirmed_at TIMESTAMPTZ,
  cancelled_at TIMESTAMPTZ,
  cancellation_reason TEXT,
  reminder_sent_at TIMESTAMPTZ,
  -- ... timestamps
);

-- reminder_events: email tracking
CREATE TABLE reminder_events (
  id UUID PRIMARY KEY,
  booking_id UUID REFERENCES bookings(id),
  channel TEXT, -- 'email', 'sms' (future)
  status reminder_status, -- pending, sent, failed, confirmed, cancelled
  scheduled_at TIMESTAMPTZ,
  sent_at TIMESTAMPTZ,
  error_message TEXT,
  -- ... timestamps
);
```

What's Next: Phase 3 (Future)

Potential enhancements:

1. **Automated Reminder Scheduling**
 - Cron job to send reminders 24h before appointments
 - Background worker or Vercel Cron
2. **SMS Reminders**
 - Twilio integration
 - Same token-based confirm/cancel via SMS
3. **Advanced Calendar Features**
 - Drag-and-drop rescheduling
 - Month view
 - Export to Google Calendar/iCal
4. **Payment Integration**
 - Stripe integration
 - Require deposit for bookings
 - Handle cancellation refunds
5. **Waitlist Management**
 - Allow clients to join waitlist for full slots
 - Auto-notify when slots open up

6. Analytics Dashboard

- No-show tracking and trends
- Revenue forecasting
- Client retention metrics
- Staff utilization reports

7. Multi-location Support

- Businesses with multiple locations
- Location-specific staff and services



Troubleshooting

“No available slots” even though staff is available

Check:

1. Staff has `availability_rules` for the selected day of week
2. No `availability_exceptions` blocking that date
3. Business hours cover the time range you’re checking
4. Service duration fits within available time

Email not sending

Check:

1. `RESEND_API_KEY` is set in `.env.local`
2. Resend account is active and verified
3. `DEFAULT_FROM_EMAIL` or business `reminder_email_from` is set
4. Check Resend dashboard for error logs

Client can’t cancel booking

Reason: Cancellation policy enforcement

Solution: Check `businesses.cancellation_hours_notice` (default 24 hours). Bookings within the notice period can’t be cancelled via the portal.

Calendar shows wrong time zone

Check: Business `timezone` field in database. All times are stored in UTC but displayed in business timezone.



API Reference

Server Actions

Availability:

```
import { getAvailableSlots, getBusinessBySlug } from '@lib/actions/availability';

const { slots } = await getAvailableSlots({
  businessSlug: 'lux-beauty',
  serviceId: 'service-uuid',
  staffMemberId: 'staff-uuid', // or null for "any available"
  date: '2026-08-01', // YYYY-MM-DD
});
```

Booking:

```
import { createPublicBooking } from '@lib/actions/public-booking';

const result = await createPublicBooking({
  businessSlug: 'lux-beauty',
  serviceId: 'service-uuid',
  staffMemberId: 'staff-uuid',
  date: '2026-08-01',
  time: '10:00',
  clientName: 'John Doe',
  clientEmail: 'john@example.com',
  clientPhone: '+1-555-1234',
  notes: 'First time client',
});
```

Client Bookings:

```
import { getClientBookings, confirmBooking, cancelBooking } from '@lib/actions/client-bookings';

const { upcoming, past } = await getClientBookings();
await confirmBooking(bookingId);
await cancelBooking(bookingId, 'Change of plans');
```

Calendar:

```
import { getBookingsByDateRange } from '@lib/actions/calendar';

const { bookings } = await getBookingsByDateRange('2026-08-01', '2026-08-07');
```

Email:

```
import { sendBookingReminder } from '@lib/email/send-reminder';

const result = await sendBookingReminder(bookingId);
```

Quality Checklist

- [x] TypeScript strict mode - zero errors
- [x] Server-side validation for all booking operations
- [x] RLS policies enforced on all tables

- [x] Mobile-responsive design
 - [x] Loading states on all async operations
 - [x] Error handling with user-friendly messages
 - [x] Security: token-based email actions
 - [x] Concurrency-safe booking creation
 - [x] No hardcoded IDs or test data in production code
 - [x] Clean component separation (UI, logic, data)
-

Testing Guide

1. Public Booking Flow:

- [] Can access `/book/luxe-beauty`
- [] Can see all 4 services
- [] Can select service and see available staff
- [] Can pick a date and see available slots
- [] Can complete booking with client info
- [] Booking appears in database

2. Client Portal:

- [] Can log in as client
- [] Can view upcoming bookings
- [] Can confirm pending booking
- [] Can cancel booking (respects policy)
- [] Can update profile

3. Email System:

- [] Can manually trigger reminder
- [] Email renders correctly
- [] Confirm link works
- [] Cancel link works
- [] Booking status updates

4. Calendar:

- [] Week view displays bookings
 - [] Color coding by service
 - [] Staff filter works
 - [] Navigation works
 - [] Bookings show details on click
-

Congratulations! You now have a fully functional Phase 2 booking system. Push to GitHub and you're ready to deploy! 🚀

```
git push origin main
```